

Usos e Exemplos dos Padrões de Projeto

Objetivo do arquivo

Este arquivo mostra exemplos práticos dos padrões de projeto estudados na disciplina.

A ideia é deixar claro quando usar cada padrão, qual problema aparece sem ele e qual ganho ele traz para o projeto.

Como estudar este arquivo

- Primeiro entenda o problema.
- Depois veja como o padrão resolve.
- Depois memorize o ganho principal.
- Em prova, sempre relacione o padrão com baixo acoplamento, alta coesão, reuso, extensibilidade ou separação de responsabilidades.

Padrões Criacionais

Factory Method

- Categoria: Criacional
- Quando usar: quando a criação de objetos varia conforme tipo, contexto ou regra de negócio.
- Situação sem o padrão: uma classe precisa criar diretamente vários tipos de objeto usando vários `if/else`.
- Exemplo ruim em TypeScript:

```
class PagamentoService {  
  criar(tipo: string) {  
    if (tipo === "cartao") return new PagamentoCartao();  
    if (tipo === "pix") return new PagamentoPix();  
    return null;  
  }  
}
```

- Problema gerado: alto acoplamento com classes concretas e dificuldade para extensão.
- Solução usando o padrão: a decisão de criação fica em um método/fábrica, e o cliente usa a abstração.
- Exemplo prático: um sistema de pedidos cria diferentes formas de pagamento, como cartão, Pix ou boleto, sem espalhar `new` pela aplicação.
- Exemplo em TypeScript:

```
interface Pagamento {  
  pagar(valor: number): void;  
}
```

```
class PagamentoCartao implements Pagamento {
    pagar(valor: number): void {}
}

class PagamentoPix implements Pagamento {
    pagar(valor: number): void {}
}

abstract class PagamentoFactory {
    abstract criar(): Pagamento;
}

class PagamentoFactoryCartao extends PagamentoFactory {
    criar(): Pagamento {
        return new PagamentoCartao();
    }
}
```

- Ganho principal: centralizar a criação e facilitar novos tipos sem alterar o cliente.
- Relação com princípios de projeto: reduz acoplamento, favorece extensão sem modificar código existente e programa para interfaces, não para implementações.
- Quando evitar: quando existe apenas um tipo de objeto e a criação é trivial.
- Frase curta para prova: "Factory Method centraliza a criação e reduz a dependência do cliente com classes concretas."

Abstract Factory

- Categoria: Criacional
- Quando usar: quando o sistema precisa criar famílias de objetos relacionados.
- Situação sem o padrão: cada parte da interface ou sistema escolhe classes concretas separadamente.
- Exemplo ruim em TypeScript:

```
class Tela {
    criarBotao() {
        return new BotaoClaro();
    }
    criarMenu() {
        return new MenuClaro();
    }
}
```

- Problema gerado: dependência de classes concretas, inconsistência entre objetos e manutenção mais difícil.
- Solução usando o padrão: uma fábrica abstrata cria produtos compatíveis da mesma família.
- Exemplo prático: um sistema de interface gráfica cria botões, menus e caixas de seleção no estilo claro ou escuro.
- Exemplo em TypeScript:

```
interface Botao {
  render(): void;
}

interface Menu {
  render(): void;
}

abstract class UIFactory {
  abstract criarBotao(): Botao;
  abstract criarMenu(): Menu;
}

class UIFactoryClaro extends UIFactory {
  criarBotao(): Botao {
    return { render() {} };
  }

  criarMenu(): Menu {
    return { render() {} };
  }
}
```

- Ganho principal: trocar uma família inteira de objetos sem mexer no código cliente.
- Relação com princípios de projeto: reduz acoplamento, aumenta coesão da família de objetos e favorece extensibilidade.
- Quando evitar: quando o sistema não precisa de famílias compatíveis, apenas de um objeto isolado.
- Frase curta para prova: "Abstract Factory cria famílias compatíveis de objetos e evita dependência direta de classes concretas."

Builder

- Categoria: Criacional
- Quando usar: quando um objeto é complexo e tem muitos passos ou opções de montagem.
- Situação sem o padrão: construtores grandes, sequência confusa de inicialização e responsabilidade misturada.
- Exemplo ruim em TypeScript:

```
class Relatorio {
  constructor(
    titulo: string,
    secoes: string[],
    cabecalho: string,
    resumo: string,
    anexos: string[],
  ) {}
}
```

- Problema gerado: baixa coesão na criação e dificuldade de manutenção.
- Solução usando o padrão: a montagem fica separada da representação final do objeto.
- Exemplo prático: um relatório acadêmico é montado com título, cabeçalho, seções, resumo e anexos em etapas.
- Exemplo em TypeScript:

```
class Relatorio {
  constructor(
    public titulo: string,
    public secoes: string[],
  ) {}
}

class RelatorioBuilder {
  private titulo = "";
  private secoes: string[] = [];

  setTitulo(titulo: string): this {
    this.titulo = titulo;
    return this;
  }

  addSecao(secao: string): this {
    this.secoes.push(secao);
    return this;
  }

  build(): Relatorio {
    return new Relatorio(this.titulo, this.secoes);
  }
}
```

- Ganho principal: separar construção e deixar a criação mais clara e controlada.
- Relação com princípios de projeto: separa responsabilidades, melhora coesão e facilita manutenção.
- Quando evitar: quando o objeto é simples e não precisa de montagem em etapas.
- Frase curta para prova: "Builder separa a montagem do objeto da sua representação e evita construtores complicados."

Prototype

- Categoria: Criacional
- Quando usar: quando é mais fácil copiar um objeto pronto do que criá-lo do zero.
- Situação sem o padrão: o sistema recria configurações parecidas várias vezes.
- Exemplo ruim em TypeScript:

```
function criarFormularioBase() {  
  return new Formulario("Cadastro", ["nome", "email", "cpf"]);  
}
```

- Problema gerado: duplicação de configuração e criação desnecessariamente trabalhosa.
- Solução usando o padrão: um protótipo serve de base para gerar novos objetos por cópia.
- Exemplo prático: um sistema de formulários cria novos formulários a partir de um modelo já preenchido com configurações padrão.
- Exemplo em TypeScript:

```
class Formulario {  
  constructor(  
    public titulo: string,  
    public campos: string[],  
  ) {}  
  
  clone(): Formulario {  
    return new Formulario(this.titulo, [...this.campos]);  
  }  
}  
  
const modelo = new Formulario("Cadastro", ["nome", "email"]);  
const copia = modelo.clone();
```

- Ganho principal: facilitar reuso da configuração inicial e reduzir custo de criação.
- Relação com princípios de projeto: favorece reuso, melhora manutenção e pode reduzir acoplamento com a lógica de montagem.
- Quando evitar: quando a cópia de estado é difícil ou quando o objeto é simples.
- Frase curta para prova: "Prototype cria novos objetos por cópia de um modelo existente, favorecendo reuso."

Singleton

- Categoria: Criacional
- Quando usar: quando existe um recurso realmente único no sistema.
- Situação sem o padrão: várias instâncias competindo pelo mesmo papel.
- Exemplo ruim em TypeScript:

```
class Logger {  
  log(_: string) {}  
}  
  
const a = new Logger();  
const b = new Logger();
```

- Problema gerado: controle disperso, inconsistência e dificuldade para compartilhar estado comum.
- Solução usando o padrão: a classe garante uma única instância e acesso global controlado.
- Exemplo prático: uma configuração global do sistema ou um logger central compartilhado por toda a aplicação.
- Exemplo em TypeScript:

```
class Logger {  
    private static instance: Logger | null = null;  
  
    private constructor() {}  
  
    static getInstance(): Logger {  
        if (!Logger.instance) Logger.instance = new Logger();  
        return Logger.instance;  
    }  
  
    log(msg: string): void {}  
}
```

- Ganho principal: concentrar o acesso a um recurso único.
- Relação com princípios de projeto: pode ajudar controle centralizado, mas pode aumentar acoplamento e prejudicar Inversão de Dependência.
- Quando evitar: quando a dependência pode ser passada normalmente por composição ou injeção.
- Frase curta para prova: "Singleton garante uma única instância, mas deve ser usado com cuidado porque pode criar acoplamento global."

Padrões Estruturais

Adapter

- Categoria: Estrutural
- Quando usar: quando duas interfaces são incompatíveis, mas você precisa integrá-las.
- Situação sem o padrão: o cliente não consegue usar uma classe existente sem adaptar manualmente chamadas e formatos.
- Exemplo ruim em TypeScript:

```
const api = new SmsServiceExterno();  
api.sendSms("mensagem");
```

- Problema gerado: acoplamento com detalhes externos e código de integração espalhado.
- Solução usando o padrão: um adaptador converte a interface antiga ou diferente para a interface esperada.
- Exemplo prático: um sistema de notificações usa um serviço externo de SMS que tem métodos diferentes dos esperados pelo sistema.
- Exemplo em TypeScript:

```
interface Notificador {
    enviar(mensagem: string): void;
}

class SmsServiceExterno {
    sendSms(texto: string): void {}
}

class SmsAdapter implements Notificador {
    constructor(private api: SmsServiceExterno) {}

    enviar(mensagem: string): void {
        this.api.sendSms(mensagem);
    }
}

const notificador: Notificador = new SmsAdapter(new SmsServiceExterno());
notificador.enviar("Olá");
```

- Ganho principal: integrar sem mudar o cliente.
- Relação com princípios de projeto: reduz acoplamento, favorece Inversão de Dependência e programa para interfaces.
- Quando evitar: quando a interface já é compatível ou quando a adaptação é desnecessária.
- Frase curta para prova: "Adapter permite usar uma classe com interface diferente sem alterar o cliente."

Bridge

- Categoria: Estrutural
- Quando usar: quando abstração e implementação podem variar independentemente.
- Situação sem o padrão: muitas subclasses para combinar variações de forma e implementação.
- Exemplo ruim em TypeScript:

```
class EmailUrgenteWindows {}
class EmailNormalWindows {}
class SmsUrgenteWindows {}
```

- Problema gerado: explosão de classes e dificuldade para extensão.
- Solução usando o padrão: separar a abstração da implementação e conectá-las por composição.
- Exemplo prático: um sistema de mensagens separa o tipo da mensagem da forma de envio, como e-mail, SMS ou push.
- Exemplo em TypeScript:

```
interface CanalEnvio {
    enviar(mensagem: string): void;
}
```

```
abstract class Mensagem {
    constructor(protected canal: CanalEnvio) {}

    abstract enviar(texto: string): void;
}

class MensagemEmail extends Mensagem {
    enviar(texto: string): void {
        this.canal.enviar(texto);
    }
}
```

- Ganho principal: variar partes independentes sem criar uma hierarquia enorme.
- Relação com princípios de projeto: reduz acoplamento, melhora reuso e favorece extensão.
- Quando evitar: quando não existem duas dimensões reais de variação.
- Frase curta para prova: "Bridge separa abstração e implementação para evitar explosão de subclasses."

Composite

- Categoria: Estrutural
- Quando usar: quando você precisa tratar item individual e grupo da mesma forma.
- Situação sem o padrão: a lógica fica cheia de casos separados para folha e para grupo.
- Exemplo ruim em TypeScript:

```
function listar(item: any) {
    if (item.tipo === "arquivo") item.abrir();
    else item.filhos.forEach(listar);
}
```

- Problema gerado: código repetitivo, baixa coesão e cliente mais complexo.
- Solução usando o padrão: componentes simples e compostos expõem a mesma interface.
- Exemplo prático: uma pasta do sistema de arquivos contém arquivos e outras pastas, mas o cliente usa a mesma operação para ambos.
- Exemplo em TypeScript:

```
interface ComponenteArquivo {
    listar(): void;
}

class Arquivo implements ComponenteArquivo {
    listar(): void {}
}

class Pasta implements ComponenteArquivo {
    constructor(private filhos: ComponenteArquivo[]) {}
    listar(): void {}
}
```



```
}

const raiz = new Pasta([new Arquivo(), new Arquivo()]);
raiz.listar();
```

- Ganho principal: tratar parte e todo de forma uniforme.
- Relação com princípios de projeto: melhora reuso, separação de responsabilidades e reduz acoplamento com a estrutura interna.
- Quando evitar: quando a estrutura não é hierárquica ou não há necessidade de tratar tudo igual.
- Frase curta para prova: "Composite trata objetos individuais e composições pela mesma interface."

Decorator

- Categoria: Estrutural
- Quando usar: quando você quer adicionar comportamento sem criar muitas subclasses.
- Situação sem o padrão: herança gera várias combinações de classes.
- Exemplo ruim em TypeScript:

```
class CafeLeite {}
class CafeLeiteChantilly {}
class CafeLeiteChantillyCanela {}
```

- Problema gerado: explosão de subclasses e baixa flexibilidade.
- Solução usando o padrão: cada novo comportamento vira um decorador empilhável por composição.
- Exemplo prático: um café pode receber leite, chantilly e canela em combinações diferentes.
- Exemplo em TypeScript:

```
interface Cafe {
  custo(): number;
}

class CafeExpresso implements Cafe {
  custo(): number {
    return 5;
  }
}

abstract class CafeDecorator implements Cafe {
  constructor(protected base: Cafe) {}
  abstract custo(): number;
}

class Leite extends CafeDecorator {
  custo(): number {
    return this.base.custo() + 2;
  }
}
```

```
const cafeComLeite = new Leite(new CafeExpresso());
```

- Ganho principal: adicionar funcionalidades sem modificar a classe original.
- Relação com princípios de projeto: favorece Aberto/Fechado, reduz acoplamento e melhora reuso de comportamentos.
- Quando evitar: quando a funcionalidade extra é fixa e muito simples.
- Frase curta para prova: "Decorator adiciona responsabilidades dinamicamente sem alterar a classe base."

Facade

- Categoria: Estrutural
- Quando usar: quando um subsistema é complexo e o cliente precisa de uma entrada simples.
- Situação sem o padrão: o cliente chama várias classes diretamente e conhece detalhes internos demais.
- Exemplo ruim em TypeScript:

```
class HomeTheater {  
  ligarTudo() {  
    new Tv().ligar();  
    new Som().ligar();  
    new Projetor().ligar();  
  }  
}
```

- Problema gerado: alto acoplamento e baixa clareza do uso.
- Solução usando o padrão: uma fachada oferece operações de alto nível e coordena o subsistema internamente.
- Exemplo prático: um sistema de home theater usa uma única interface para ligar TV, som, projetor e luzes.
- Exemplo em TypeScript:

```
class HomeTheaterFacade {  
  ligarTudo(): void {}  
  assistirFilme(): void {}  
}  
  
const cinema = new HomeTheaterFacade();  
cinema.assistirFilme();
```

- Ganho principal: simplificar o uso e esconder complexidade.
- Relação com princípios de projeto: reduz acoplamento, melhora encapsulamento e separa responsabilidades.
- Quando evitar: quando o subsistema já é simples ou quando a fachada viraria uma classe "faz tudo".

- Frase curta para prova: "Facade simplifica o acesso a um subsistema e reduz o acoplamento do cliente."

Flyweight

- Categoria: Estrutural
- Quando usar: quando há muitos objetos muito parecidos e você quer economizar memória.
- Situação sem o padrão: criar uma instância para cada variação pequena de estado.
- Exemplo ruim em TypeScript:

```
const a = new CharacterFlyweight("a");
const b = new CharacterFlyweight("a");
const c = new CharacterFlyweight("a");
```

- Problema gerado: desperdício de memória e duplicação de dados repetidos.
- Solução usando o padrão: compartilhar estado comum e deixar só o estado variável fora do objeto.
- Exemplo prático: um editor de texto pode reutilizar estruturas de caracteres repetidos em vez de duplicar tudo.
- Exemplo em TypeScript:

```
class CharacterFlyweight {
  constructor(public simbolo: string) {}
}

class FlyweightFactory {
  private cache = new Map<string, CharacterFlyweight>();

  get(simbolo: string): CharacterFlyweight {
    return this.cache.get(simbolo) ?? new CharacterFlyweight(simbolo);
  }
}

const factory = new FlyweightFactory();
const charA = factory.get("a");
```

- Ganho principal: reduzir consumo de memória e reaproveitar estado compartilhado.
- Relação com princípios de projeto: favorece reuso e melhora eficiência, embora aumente complexidade interna.
- Quando evitar: quando o número de objetos é pequeno ou a economia de memória não compensa.
- Frase curta para prova: "Flyweight compartilha estado para reduzir memória quando há muitos objetos semelhantes."

Proxy

- Categoria: Estrutural
- Quando usar: quando o acesso a um objeto precisa ser controlado.
- Situação sem o padrão: o cliente acessa o objeto real diretamente, sem controle, cache ou proteção.

- Exemplo ruim em TypeScript:

```
const documento = new DocumentoReal();
documento.abrir();
```

- Problema gerado: acoplamento direto, falta de controle e responsabilidades misturadas.
- Solução usando o padrão: o proxy atua como intermediário entre cliente e objeto real.
- Exemplo prático: um serviço de documentos usa proxy para validar acesso, registrar log ou adiar carregamento do arquivo.
- Exemplo em TypeScript:

```
interface Documento {
  abrir(): void;
}

class DocumentoReal implements Documento {
  abrir(): void {}
}

class DocumentoProxy implements Documento {
  constructor(private real: DocumentoReal) {}
  abrir(): void {}
}

const doc: Documento = new DocumentoProxy(new DocumentoReal());
doc.abrir();
```

- Ganho principal: controlar acesso sem expor o objeto real ao cliente.
- Relação com princípios de projeto: reduz acoplamento, separa responsabilidades e pode melhorar encapsulamento.
- Quando evitar: quando não há necessidade de controle extra sobre o acesso.
- Frase curta para prova: "Proxy controla o acesso a um objeto real e pode adicionar cache, validação ou log."

Padrões Comportamentais

Chain of Responsibility

- Categoria: Comportamental
- Quando usar: quando uma requisição pode ser tratada por várias regras em sequência.
- Situação sem o padrão: uma classe central decide tudo com muitos `if/else`.
- Exemplo ruim em TypeScript:

```
function tratar(tipo: string) {
  if (tipo === "financeiro") {
  } else if (tipo === "suporte") {
```

```
} else if (tipo === "adm") {  
}  
}
```

- Problema gerado: alto acoplamento, baixa coesão e dificuldade de extensão.
- Solução usando o padrão: a requisição passa por uma cadeia até que algum objeto a trate.
- Exemplo prático: um sistema de atendimento classifica uma solicitação por nível de suporte, financeiro ou administrativo.
- Exemplo em TypeScript:

```
interface Handler {  
  setNext(next: Handler): Handler;  
  handle(requisicao: string): void;  
}  
  
abstract class BaseHandler implements Handler {  
  constructor(protected next?: Handler) {}  
  
  setNext(next: Handler): Handler {  
    this.next = next;  
    return next;  
  }  
  
  handle(requisicao: string): void {  
    this.next?.handle(requisicao);  
  }  
}  
  
class HandlerSuporte extends BaseHandler {  
  handle(requisicao: string): void {  
    if (requisicao === "suporte") return;  
    super.handle(requisicao);  
  }  
}
```

- Ganho principal: distribuir a responsabilidade de tratamento.
- Relação com princípios de projeto: separa responsabilidades, reduz acoplamento e facilita extensão.
- Quando evitar: quando existe apenas um único ponto de decisão.
- Frase curta para prova: "Chain of Responsibility distribui o tratamento da requisição em uma cadeia de objetos."

Command

- Categoria: Comportamental
- Quando usar: quando uma ação precisa ser tratada como objeto.
- Situação sem o padrão: a interface chama diretamente métodos específicos em vários lugares.
- Exemplo ruim em TypeScript:

```
class Tela {  
  salvar(): void {}  
}  
  
function clicarSalvar() {  
  new Tela().salvar();  
}
```

- Problema gerado: acoplamento entre quem pede a ação e quem executa.
- Solução usando o padrão: a operação é encapsulada em um comando executável.
- Exemplo prático: botões de uma interface gráfica executam ações como salvar, imprimir ou desfazer.
- Exemplo em TypeScript:

```
interface Command {  
  execute(): void;  
}  
  
class SalvarCommand implements Command {  
  execute(): void {}  
}  
  
class Botao {  
  constructor(private command: Command) {}  
  clicar(): void {  
    this.command.execute();  
  }  
}  
  
const botaoSalvar = new Botao(new SalvarCommand());  
botaoSalvar.clicar();
```

- Ganho principal: desacoplar solicitante e executor.
- Relação com princípios de projeto: reduz acoplamento, facilita reuso e separa responsabilidades.
- Quando evitar: quando a ação é muito simples e não precisa de encapsulamento.
- Frase curta para prova: "Command encapsula uma ação como objeto e desacopla quem pede de quem executa."

Interpreter

- Categoria: Comportamental
- Quando usar: quando você precisa interpretar uma gramática ou expressão simples.
- Situação sem o padrão: a lógica de interpretação fica espalhada e difícil de manter.
- Exemplo ruim em TypeScript:

```
function interpretar(expr: string) {  
  if (expr.includes("aprovado") && expr.includes("frequente")) return true;
```

```
    return false;
}
```

- Problema gerado: código confuso, baixa coesão e dificuldade para extensão.
- Solução usando o padrão: regras da gramática viram estruturas interpretáveis.
- Exemplo prático: um sistema acadêmico interpreta uma consulta simples como "aluno aprovado e frequente".
- Exemplo em TypeScript:

```
interface Expressao {
    interpretar(contexto: string): boolean;
}

class ExpressaoSimples implements Expressao {
    interpretar(contexto: string): boolean {
        return contexto.includes("aprovado");
    }
}

const expr: Expressao = new ExpressaoSimples();
expr.interpretar("aluno aprovado");
```

- Ganho principal: organizar a interpretação em regras separadas.
- Relação com princípios de projeto: melhora separação de responsabilidades, embora possa aumentar complexidade.
- Quando evitar: quando a linguagem é complexa demais para esse tipo de solução.
- Frase curta para prova: "Interpreter representa uma gramática e interpreta expressões simples dessa gramática."

Iterator

- Categoria: Comportamental
- Quando usar: quando você quer percorrer coleções sem expor sua estrutura interna.
- Situação sem o padrão: o cliente conhece detalhes da coleção e percorre por acesso direto.
- Exemplo ruim em TypeScript:

```
class Turma {
    alunos: string[] = [];
}

const turma = new Turma();
for (let i = 0; i < turma.alunos.length; i++) {}
```

- Problema gerado: acoplamento com a estrutura interna e pouca flexibilidade.
- Solução usando o padrão: um iterador padroniza o acesso sequencial aos elementos.

- Exemplo prático: listar alunos matriculados sem revelar como a turma armazena os dados internamente.
- Exemplo em TypeScript:

```
interface Iterator<T> {  
  next(): T | null;  
}  
  
class TurmaIterator implements Iterator<string> {  
  next(): string | null {  
    return null;  
  }  
}  
  
const it: Iterator<string> = new TurmaIterator();  
it.next();
```

- Ganho principal: percorrer de forma uniforme.
- Relação com princípios de projeto: melhora encapsulamento, reduz acoplamento e favorece reuso.
- Quando evitar: quando a linguagem já oferece iteração simples e suficiente.
- Frase curta para prova: "Iterator permite percorrer uma coleção sem expor sua estrutura interna."

Mediator

- Categoria: Comportamental
- Quando usar: quando muitos objetos precisam se comunicar e ficam fortemente acoplados.
- Situação sem o padrão: cada objeto conhece vários outros objetos diretamente.
- Exemplo ruim em TypeScript:

```
class Usuario {  
  mandarPara(outro: Usuario, msg: string): void {}  
}
```

- Problema gerado: alto acoplamento e manutenção difícil.
- Solução usando o padrão: um mediador centraliza a comunicação entre os colegas.
- Exemplo prático: em um sistema de chat, o mediador coordena envio de mensagens entre usuários e salas.
- Exemplo em TypeScript:

```
interface Mediator {  
  enviar(mensagem: string, remetente: object): void;  
}  
  
class ChatMediator implements Mediator {  
  enviar(mensagem: string, remetente: object): void {}  
}
```



```
const chat: Mediator = new ChatMediator();
chat.enviar("oi", {});
```

- Ganho principal: reduzir dependências diretas entre objetos.
- Relação com princípios de projeto: reduz acoplamento, separa responsabilidades e melhora organização.
- Quando evitar: quando a comunicação é pequena e direta.
- Frase curta para prova: "Mediator centraliza a comunicação e evita que os objetos dependam uns dos outros diretamente."

Memento

- Categoria: Comportamental
- Quando usar: quando o sistema precisa salvar e restaurar estado anterior.
- Situação sem o padrão: a lógica de histórico fica espalhada e o objeto expõe detalhes internos demais.
- Exemplo ruim em TypeScript:

```
class Editor {
  public estado = "";
}
```

- Problema gerado: quebra de encapsulamento e dificuldade para desfazer ações.
- Solução usando o padrão: o objeto cria registros do estado e um cuidador guarda esses registros.
- Exemplo prático: um editor de texto salva versões anteriores para permitir desfazer alterações.
- Exemplo em TypeScript:

```
class Memento {
  constructor(public readonly estado: string) {}
}

class Editor {
  criarMemento(): Memento {
    return new Memento("estado atual");
  }
}

const editor = new Editor();
const snapshot = editor.criarMemento();
```

- Ganho principal: restaurar estado sem expor o interior do objeto.
- Relação com princípios de projeto: preserva encapsulamento e separa responsabilidades.
- Quando evitar: quando não há necessidade de histórico ou restauração.
- Frase curta para prova: "Memento guarda estado anterior para permitir restauração sem quebrar encapsulamento."

Observer

- Categoria: Comportamental
- Quando usar: quando um estado central precisa notificar vários interessados.
- Situação sem o padrão: a classe central precisa conhecer e chamar manualmente cada dependente.
- Exemplo ruim em TypeScript:

```
class Boletim {  
  atualizar(): void {  
    new Aluno().notify();  
    new Professor().notify();  
  }  
}
```

- Problema gerado: alto acoplamento e dificuldade para adicionar novas visualizações ou reações.
- Solução usando o padrão: o sujeito mantém observadores e os notifica quando muda.
- Exemplo prático: o sistema de notas avisa aluno, professor e coordenação quando a situação muda.
- Exemplo em TypeScript:

```
interface Observer {  
  update(): void;  
}  
  
class Subject {  
  private observers: Observer[] = [];  
  
  addObserver(observer: Observer): void {  
    this.observers.push(observer);  
  }  
}  
  
class AlunoView implements Observer {  
  update(): void {}  
}  
  
const boletim = new Subject();  
boletim.addObserver(new AlunoView());
```

- Ganho principal: adicionar novos observadores sem mexer no sujeito.
- Relação com princípios de projeto: reduz acoplamento, favorece Aberto/Fechado e Inversão de Dependência.
- Quando evitar: quando existe apenas um consumidor da mudança.
- Frase curta para prova: "Observer notifica vários objetos quando um estado muda, sem acoplar o sujeito às classes concretas."

State

- Categoria: Comportamental
- Quando usar: quando o comportamento muda conforme o estado interno.
- Situação sem o padrão: muitos `if/else` decidem o comportamento com base no estado.
- Exemplo ruim em TypeScript:

```
function atender(estado: string) {  
  if (estado === "espera") {  
  } else if (estado === "em_chamada") {  
  }  
}
```

- Problema gerado: código rígido, difícil de ler e difícil de estender.
- Solução usando o padrão: cada estado vira um objeto e o contexto delega para ele.
- Exemplo prático: uma máquina de atendimento muda entre "livre", "em chamada" e "em pausa".
- Exemplo em TypeScript:

```
interface EstadoAtendimento {  
  atender(): void;  
}  
  
class EmEspera implements EstadoAtendimento {  
  atender(): void {}  
}  
  
class Atendimento {  
  constructor(private estado: EstadoAtendimento) {}  
}  
  
const atendimento = new Atendimento(new EmEspera());
```

- Ganho principal: encapsular o comportamento de cada estado separadamente.
- Relação com princípios de projeto: reduz acoplamento, melhora coesão e favorece extensão.
- Quando evitar: quando há poucos estados e a regra é simples.
- Frase curta para prova: "State troca o comportamento de um objeto conforme seu estado interno."

Strategy

- Categoria: Comportamental
- Quando usar: quando existem vários algoritmos possíveis para a mesma tarefa.
- Situação sem o padrão: a classe principal escolhe tudo com condicionais e precisa ser alterada para trocar a regra.
- Exemplo ruim em TypeScript:

```
class Frete {  
  calcular(tipo: string): number {  
    if (tipo === "peso") return 10;
```

```
    if (tipo === "distancia") return 20;
    return 0;
  }
}
```

- Problema gerado: baixo reaproveitamento, alto acoplamento e pouca extensibilidade.
- Solução usando o padrão: cada algoritmo vira uma estratégia separada.
- Exemplo prático: um sistema de frete escolhe entre cálculo por distância, peso ou prazo.
- Exemplo em TypeScript:

```
interface FreteStrategy {
  calcular(valor: number): number;
}

class FretePeso implements FreteStrategy {
  calcular(valor: number): number {
    return valor * 0.1;
  }
}

class Carrinho {
  constructor(private frete: FreteStrategy) {}
}

const carrinho = new Carrinho(new FretePeso());
```

- Ganho principal: trocar o algoritmo sem alterar a classe que o usa.
- Relação com princípios de projeto: reduz acoplamento, favorece Aberto/Fechado e programa para interfaces.
- Quando evitar: quando o algoritmo é fixo e não deve variar.
- Frase curta para prova: "Strategy encapsula algoritmos intercambiáveis e permite trocar o comportamento sem alterar o cliente."

Template Method

- Categoria: Comportamental
- Quando usar: quando várias classes seguem o mesmo fluxo geral, mas mudam alguns passos.
- Situação sem o padrão: o mesmo algoritmo é repetido em várias classes com pequenas diferenças.
- Exemplo ruim em TypeScript:

```
class RelatorioAluno {  
    gerar(): void {}  
}  
  
class RelatorioProfessor {  
    gerar(): void {}  
}
```

- Problema gerado: duplicação de código e baixa organização do fluxo.
- Solução usando o padrão: a classe base define o esqueleto e as subclasses ajustam etapas específicas.
- Exemplo prático: o processamento de relatório segue sempre a mesma ordem, mas muda a forma de coletar os dados.
- Exemplo em TypeScript:

```
abstract class RelatorioTemplate {  
    gerar(): void {  
        this.coletarDados();  
        this.montar();  
    }  
  
    protected abstract coletarDados(): void;  
    protected montar(): void {}  
}  
  
class RelatorioAluno extends RelatorioTemplate {  
    protected coletarDados(): void {}  
}  
  
new RelatorioAluno().gerar();
```

- Ganho principal: reaproveitar o fluxo comum e variar só os passos necessários.
- Relação com princípios de projeto: melhora reuso, separação de responsabilidades e pode ajudar Inversão de Controle.
- Quando evitar: quando as variações são muitas e o uso de herança fica rígido.
- Frase curta para prova: "Template Method fixa o fluxo principal e deixa as etapas variáveis para as subclasses."

Visitor

- Categoria: Comportamental
- Quando usar: quando a hierarquia de objetos é estável, mas novas operações aparecem com frequência.
- Situação sem o padrão: cada nova operação exige alterar todas as classes da hierarquia.
- Exemplo ruim em TypeScript:

```
class Aluno {  
    calcularMedia(): number {  
        return 0;  
    }  
}
```

- Problema gerado: dificuldade para extensão e possível quebra de encapsulamento.
- Solução usando o padrão: as operações ficam em visitantes separados e a hierarquia só aceita o visitante.
- Exemplo prático: em um sistema de cadastro acadêmico, um visitante calcula média, outro gera PDF e outro exporta dados.
- Exemplo em TypeScript:

```
interface Visitor {  
    visitAluno(aluno: Aluno): void;  
}  
  
class Aluno {  
    accept(visitor: Visitor): void {  
        visitor.visitAluno(this);  
    }  
}  
  
class PrintVisitor implements Visitor {  
    visitAluno(aluno: Aluno): void {}  
}  
  
new Aluno().accept(new PrintVisitor());
```

- Ganho principal: adicionar novas operações sem mexer nas classes visitadas.
- Relação com princípios de projeto: favorece Aberto/Fechado para novas operações, mas exige cuidado com encapsulamento.
- Quando evitar: quando a hierarquia muda com frequência ou o acesso aos dados internos é problemático.
- Frase curta para prova: "Visitor separa operações da estrutura visitada e facilita adicionar novas funcionalidades."

Tabela-resumo dos exemplos

Padrão	Exemplo usado	Problema sem o padrão	Ganho principal
Factory Method	Criação de formas de pagamento	<code>if/else</code> e acoplamento com classes concretas	Centraliza criação
Abstract Factory	Tema claro/escuro em interface gráfica	Famílias incompatíveis e manutenção difícil	Cria famílias compatíveis

Padrão	Exemplo usado	Problema sem o padrão	Ganho principal
Builder	Montagem de relatório acadêmico	Construtores confusos e muitos passos	Separa montagem
Prototype	Formulário baseado em modelo padrão	Repetição de configuração	Reuso por cópia
Singleton	Configuração global ou logger	Múltiplas instâncias e acesso disperso	Instância única
Adapter	Serviço externo de SMS	Interface incompatível	Integra sem mudar cliente
Bridge	Tipos de mensagem e canais de envio	Explosão de subclasses	Separa variações
Composite	Pasta e arquivos	Lógica separada para parte e todo	Trata estrutura uniformemente
Decorator	Café com adicionais	Herança com combinações demais	Adiciona comportamento por composição
Facade	Home theater	Cliente conhece classes internas demais	Simplifica o acesso
Flyweight	Muitos caracteres repetidos	Desperdício de memória	Compartilha estado
Proxy	Documento com controle de acesso	Acesso direto sem controle	Intercepta e controla acesso
Chain of Responsibility	Triagem de atendimento	Muitos <code>if/else</code> em uma classe	Distribui o tratamento
Command	Botões da interface	Acoplamento entre ação e executor	Encapsula ação
Interpreter	Consulta simples em sistema acadêmico	Regras espalhadas e difíceis de manter	Organiza gramática
Iterator	Listar alunos de uma turma	Estrutura exposta ao cliente	Percurso uniforme
Mediator	Chat entre usuários	Objetos fortemente interligados	Centraliza comunicação
Memento	Histórico de alterações em editor	Desfazer sem histórico organizado	Restaura estado
Observer	Mudança de situação de aluno	Notificação manual e acoplada	Notifica vários observadores
State	Máquina com estados de operação	Muitos condicionais de estado	Encapsula comportamento por

Padrão	Exemplo usado	Problema sem o padrão	Ganho principal
			estado
Strategy	Cálculo de frete	Troca de algoritmo exige alterar a classe	Troca comportamento facilmente
Template Method	Processamento de relatório	Fluxo repetido em várias classes	Reaproveita esqueleto
Visitor	Operações sobre cadastro acadêmico	Cada operação altera a hierarquia	Adiciona operações sem mudar classes

Exemplos mais fáceis de lembrar na prova

- Factory Method: criar formas de pagamento sem acoplar o cliente à classe concreta.
- Abstract Factory: criar tema claro ou escuro de interface.
- Builder: montar relatório ou documento em etapas.
- Prototype: copiar formulário ou modelo pronto.
- Singleton: logger ou configuração global.
- Adapter: adaptar serviço antigo ou API externa.
- Bridge: separar tipo da mensagem e canal de envio.
- Composite: pasta e arquivos.
- Decorator: adicionar adicionais a um café.
- Facade: controlar um home theater com uma única interface.
- Flyweight: reutilizar caracteres ou objetos repetidos.
- Proxy: controlar acesso a um documento ou serviço.
- Chain of Responsibility: triagem de atendimento.
- Command: botões de interface executando ações.
- Interpreter: interpretar consulta ou expressão simples.
- Iterator: percorrer alunos de uma turma.
- Mediator: coordenação de chat ou sala de aula.
- Memento: desfazer alterações em editor.
- Observer: notificar mudanças de status.
- State: máquina com estados.
- Strategy: escolher forma de cálculo de frete.
- Template Method: fluxo fixo com passos variáveis.
- Visitor: novas operações sobre cadastro ou hierarquia estável.

Como responder questões práticas

1. Identifique o problema de projeto.
2. Diga qual padrão resolve.
3. Explique como o padrão organiza as classes.
4. Cite o ganho usando termos de projeto.
5. Cite uma possível desvantagem.